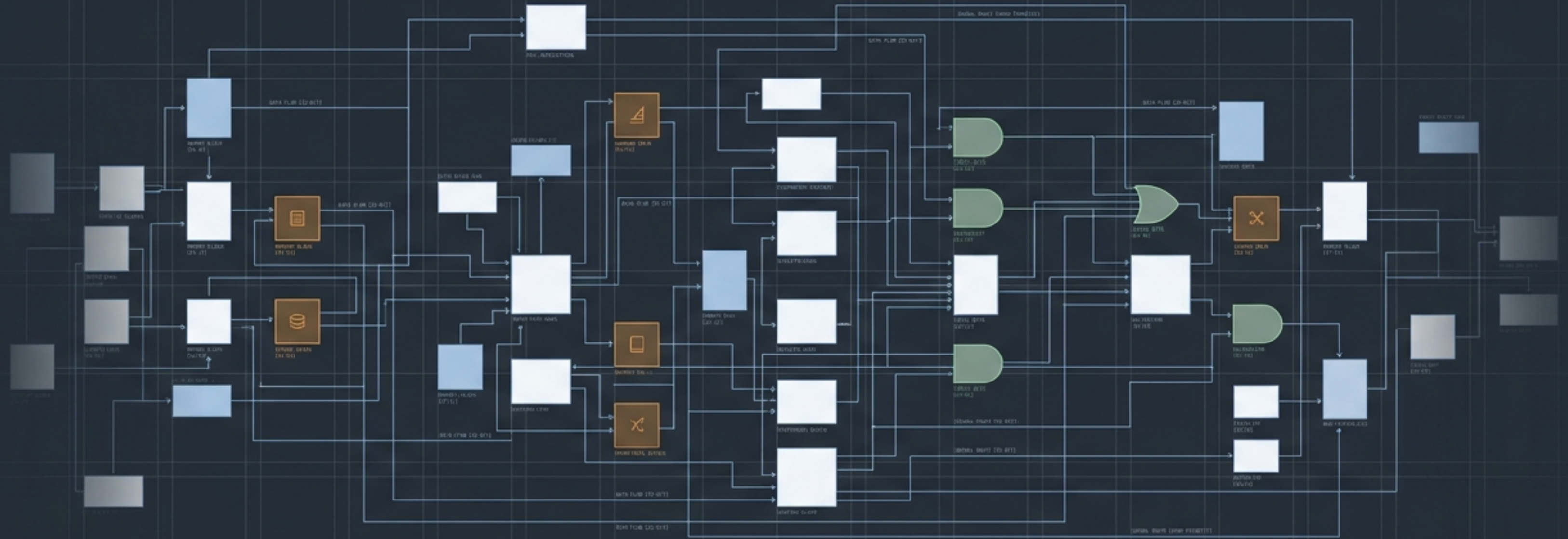




Building Agentic Architecture

A structural blueprint of LangGraph's core concepts.

FROM LINEAR SCRIPTS TO STATE MACHINES

STANDARD LLM SCRIPT

```
def data_event():
    text = test
    sodes = new data[]
    if exepsttens == data:
        return sodes.state)
    else:
        return someporsssoent)

    if (datais:
        return sveanything(ist")

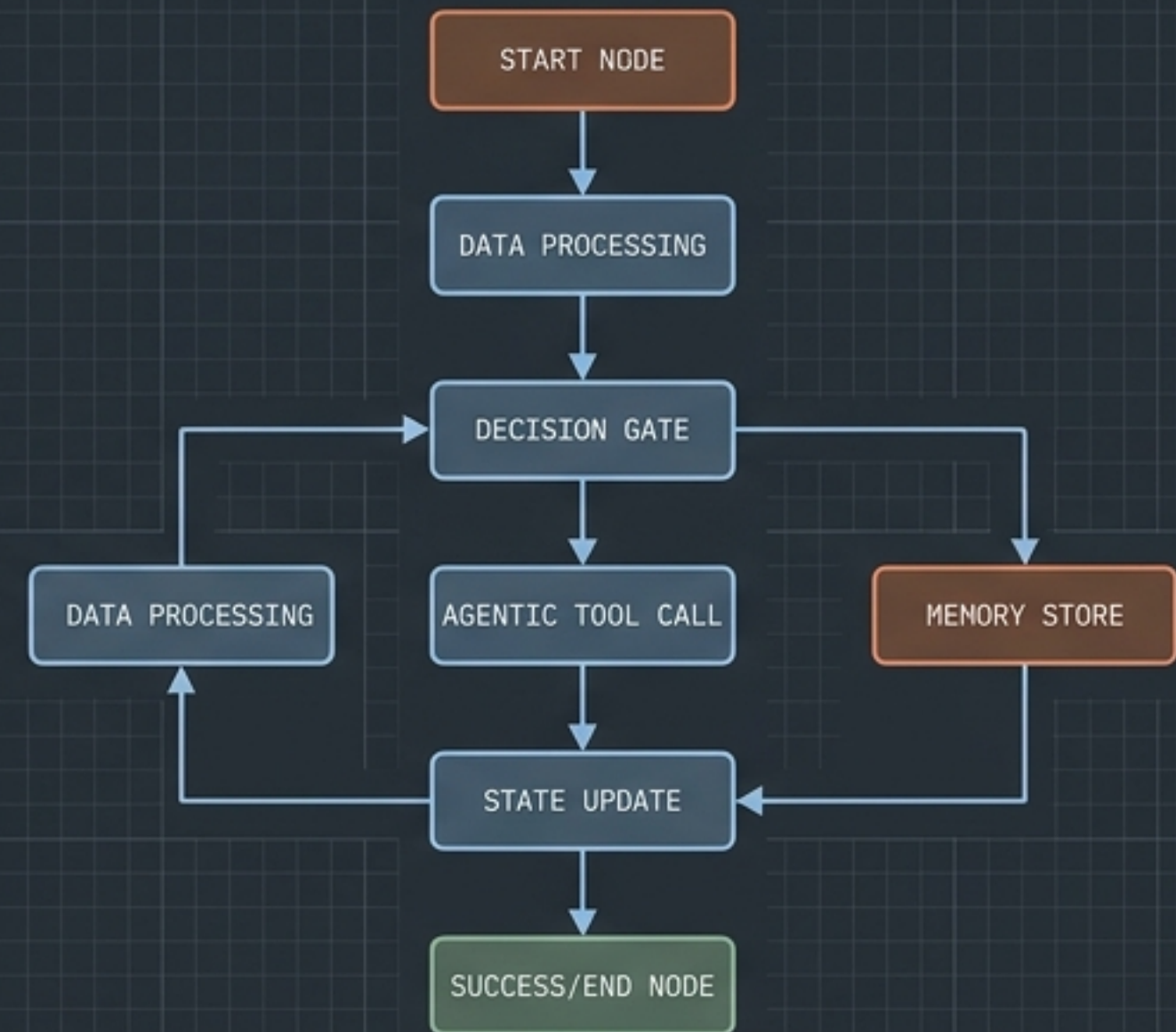
def agent_essence():
    """teute_eosenee():
    tesp = []
    tasks = oows:losast.event()
    while iinverepos:
        state = asonteke)
        if n = 8:
            woufceBantlag("suner")
        else:
            return akeontPassene()

def detaIdeout():
    rars.peecesssn()
    assuit = noeassr.events()
    data.steve
    if eodess:
        self.statesietingan
        self.ereeseution = execute()
    else:
        return simplietmlloge()

def send_snd_load:
    statstime.prog()
    sessory = etmrecessey.printf()
    nodeam = "Sutorina")
    azecote = []
    for wsoes:
        stote.response.getPan()
        return mtaek.nodsabs
```

Standard LLM Script

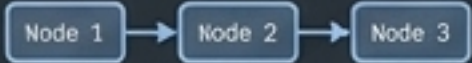
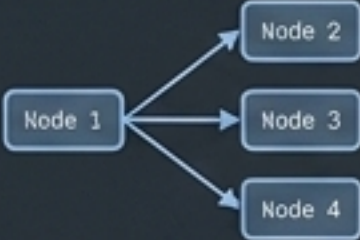
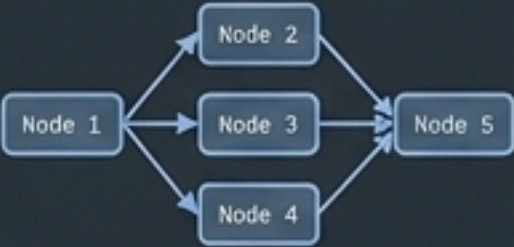
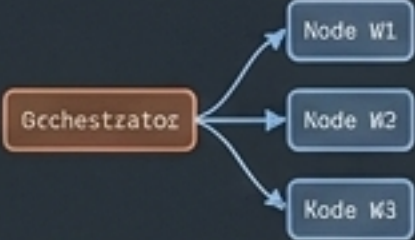
LANGGRAPH ORCHESTRATION



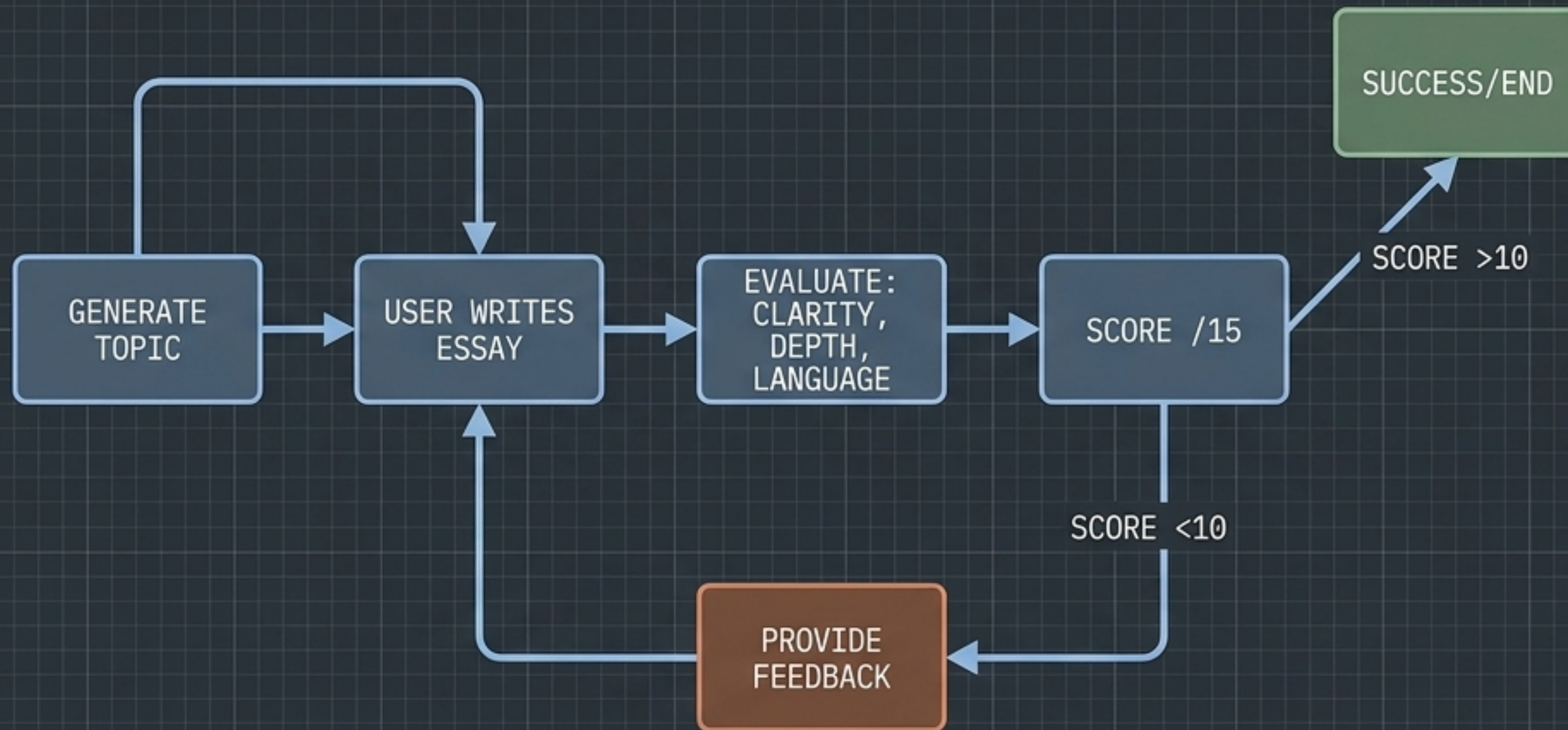
LangGraph Orchestration

LangGraph is an **orchestration framework**. It transforms complex LLM workflows into structured graphs, bringing deterministic execution to agentic systems. By representing tasks as interconnected nodes, it enables **parallel** execution, **cyclic loops**, **branching logic**, and **persistent memory**—the foundations of production-grade AI.

ARCHETYPAL WORKFLOW TOPOLOGIES

PATTERN	TOPOLOGY TYPE	DECISION LOGIC	PRODUCTION USE CASE
Prompt Chaining 	Linear	Fixed sequential	Topic -> Outline -> Report (with length checks, e.g., <5000 words).
Routing 	Branching	Classifier-driven	Customer Support (routing to Refund, Tech, or Sales LLMs).
Parallelization 	Fork & Join	Concurrent	YouTube Content Moderation (simultaneously checking Guidelines, Misinformation, Sexual Content).
Orchestrator-Worker 	Dynamic Fork	Input-dependent	Research Assistant (routing to Google Scholar for tech vs. Google News for politics).
Evaluator-Optimizer 	Cyclic Loop	Criteria-based	Creative generation (Drafting -> Evaluator checks -> Rejects with feedback -> Loop until accepted).

ANATOMY OF A GRAPH: THE UPSC EVALUATOR



GRAPH ELEMENTS



Nodes: Behind the scenes, every node is simply a Python function. They define what work is done.



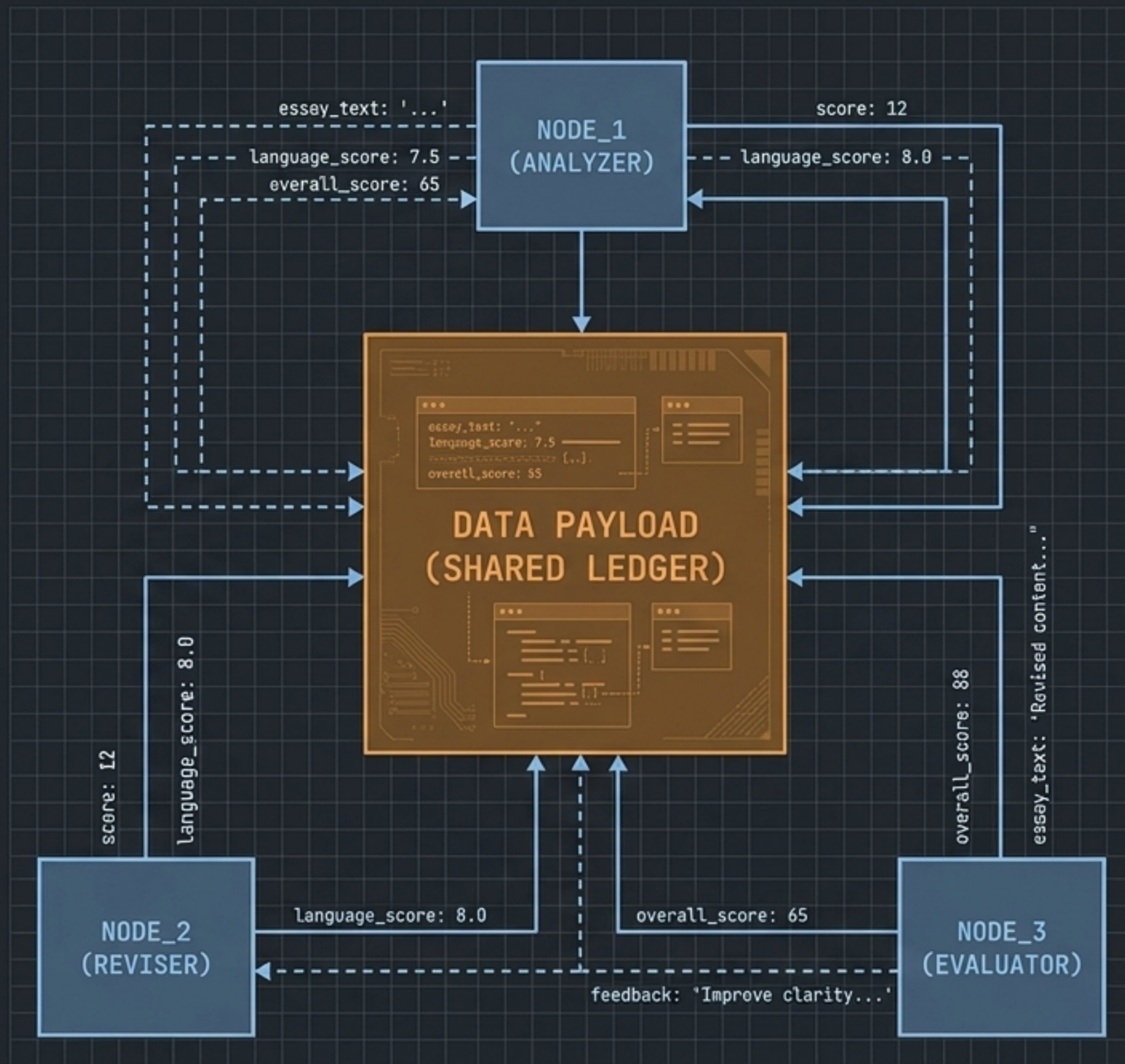
Edges: Directional pathways that define when and how execution flows (Sequential, Parallel, Conditional, Looping).

The Central Nervous System: State

An LLM workflow requires data points that evolve over time. In LangGraph, State is a shared, mutable memory that flows through the workflow.

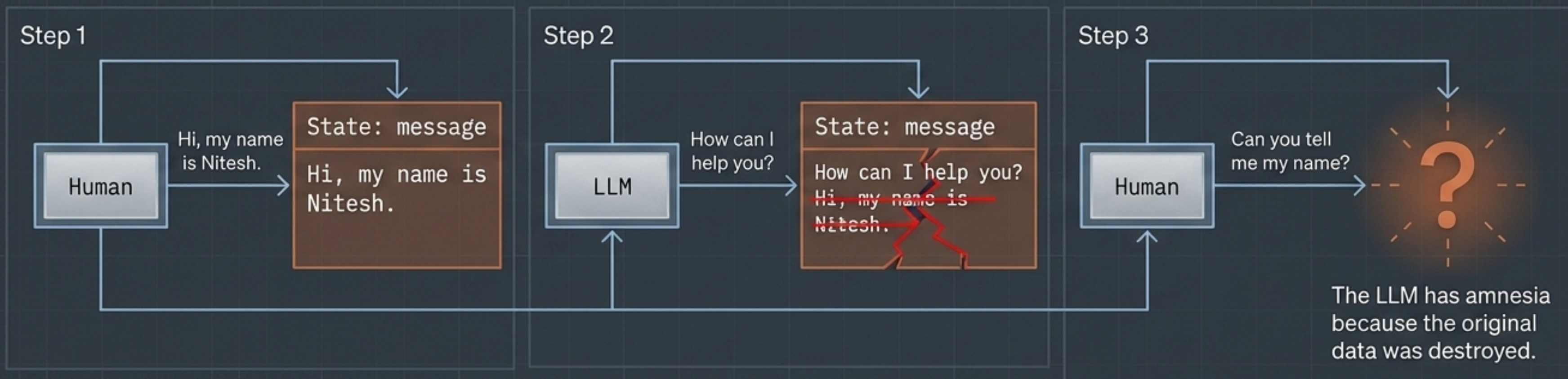
TECHNICAL SPECIFICATIONS

- Accessible to all nodes at any moment.
- Typically defined as a TypedDict or Pydantic object.
- Holds key-value pairs (e.g., `essay_text`, `language_score`, `overall_score`).
- Nodes receive the full state as input, perform tasks, and return partial updates.



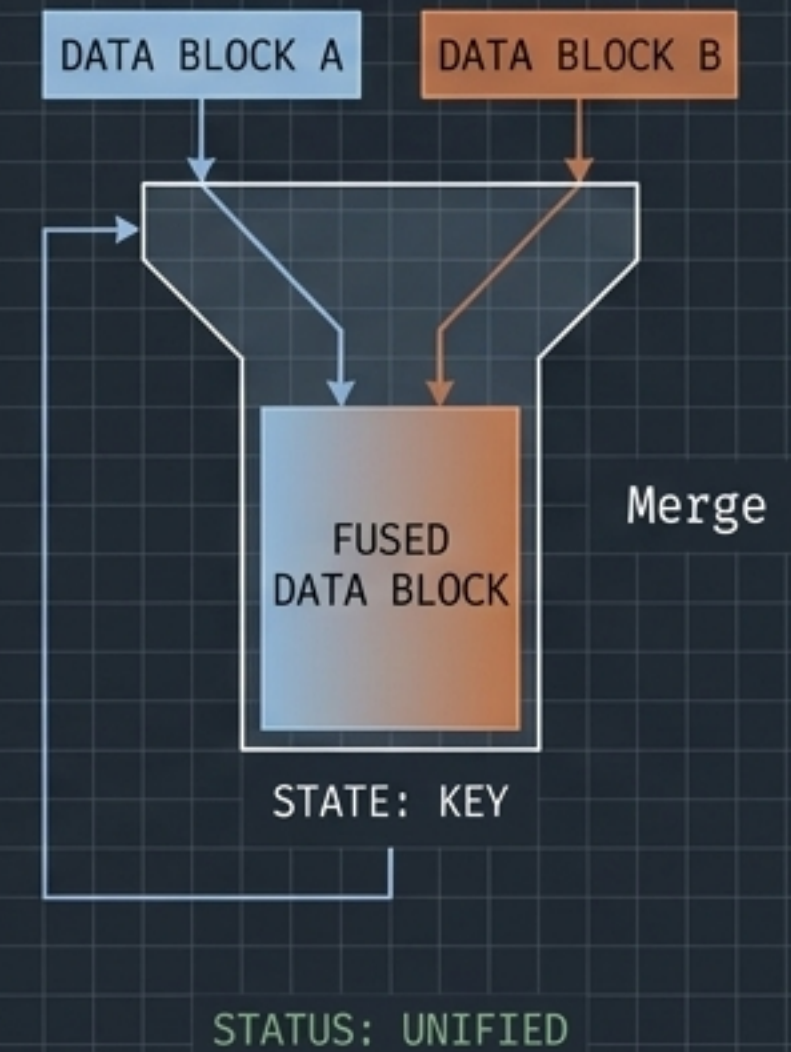
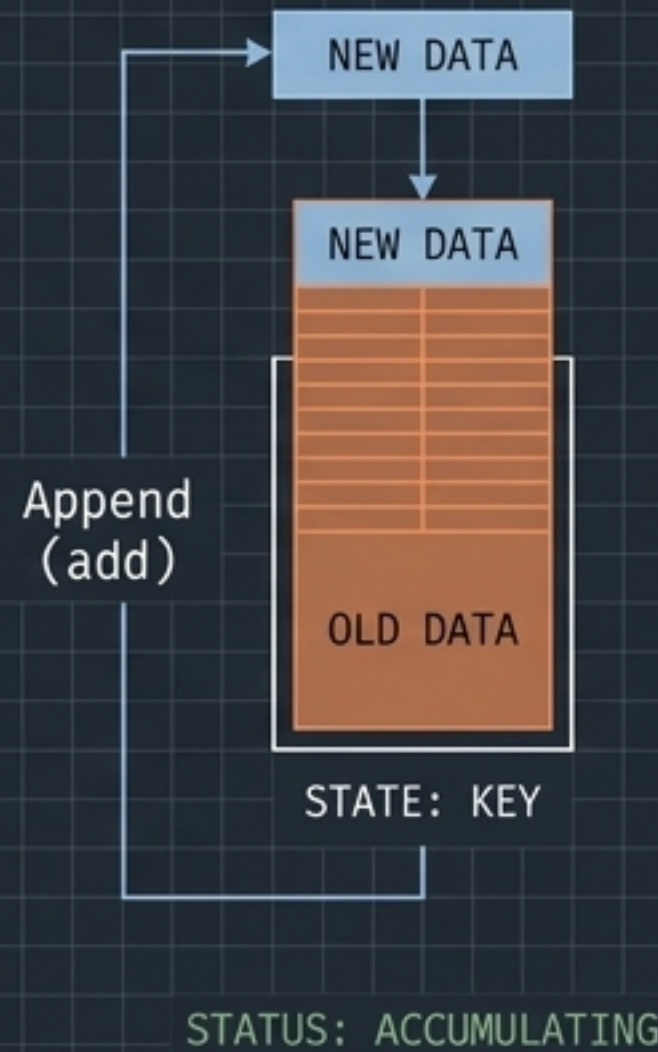
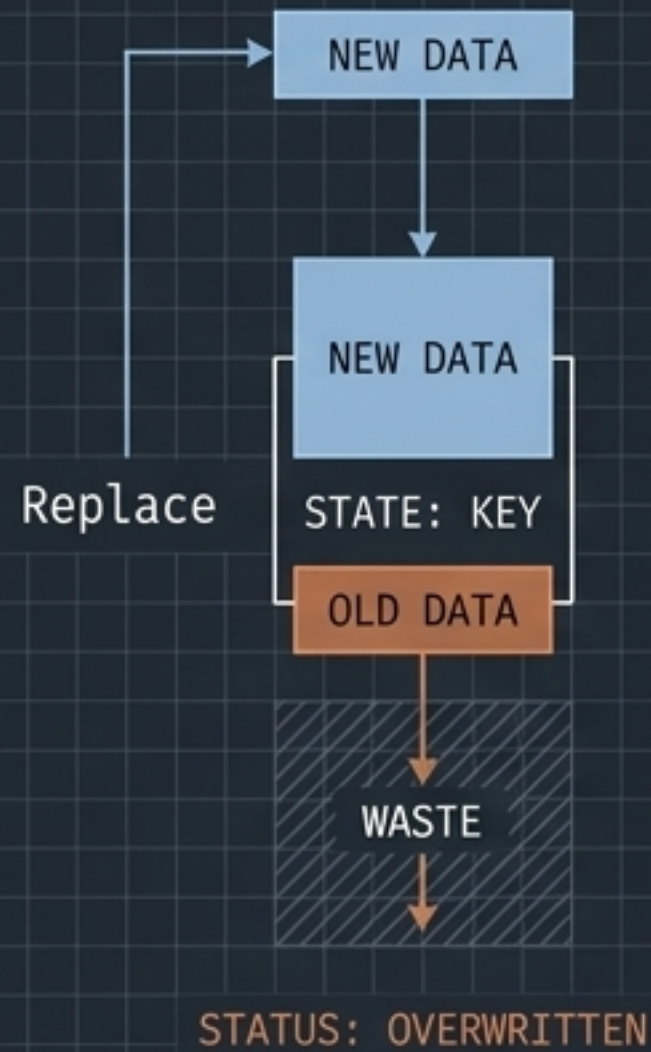
The State Mutation Problem

By default, nodes overwrite existing keys in the state. For basic workflows (e.g., updating a math result from 11 to 22), this works. But for conversational agents or iterative drafting, overwriting destroys critical historical context.



Logic Valves: Reducers

Reducers define how node updates are applied to the shared state. Each key in your state can have its own customized reducer.



THE UPSC SOLUTION

Instead of overwriting a student's essay upon a retry, assigning an 'add' reducer to the 'essay_text' key preserves all drafts, allowing the system to track the student's evolution over multiple attempts.

Pre-Flight Integrity: Compilation

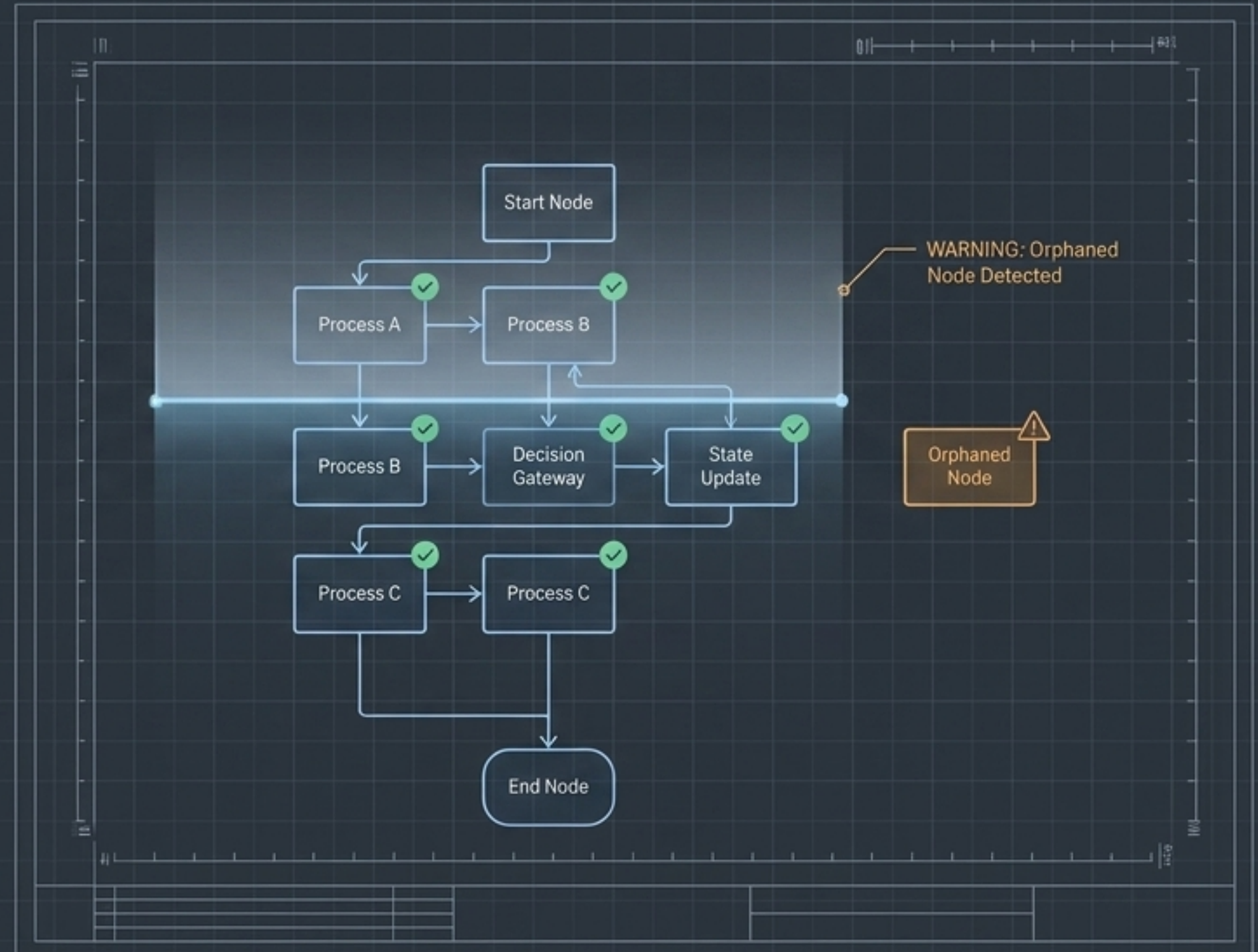
Before a graph can execute, it must be defined and compiled.



1. **Define:** Establish Nodes, Edges, and the State TypedDict.

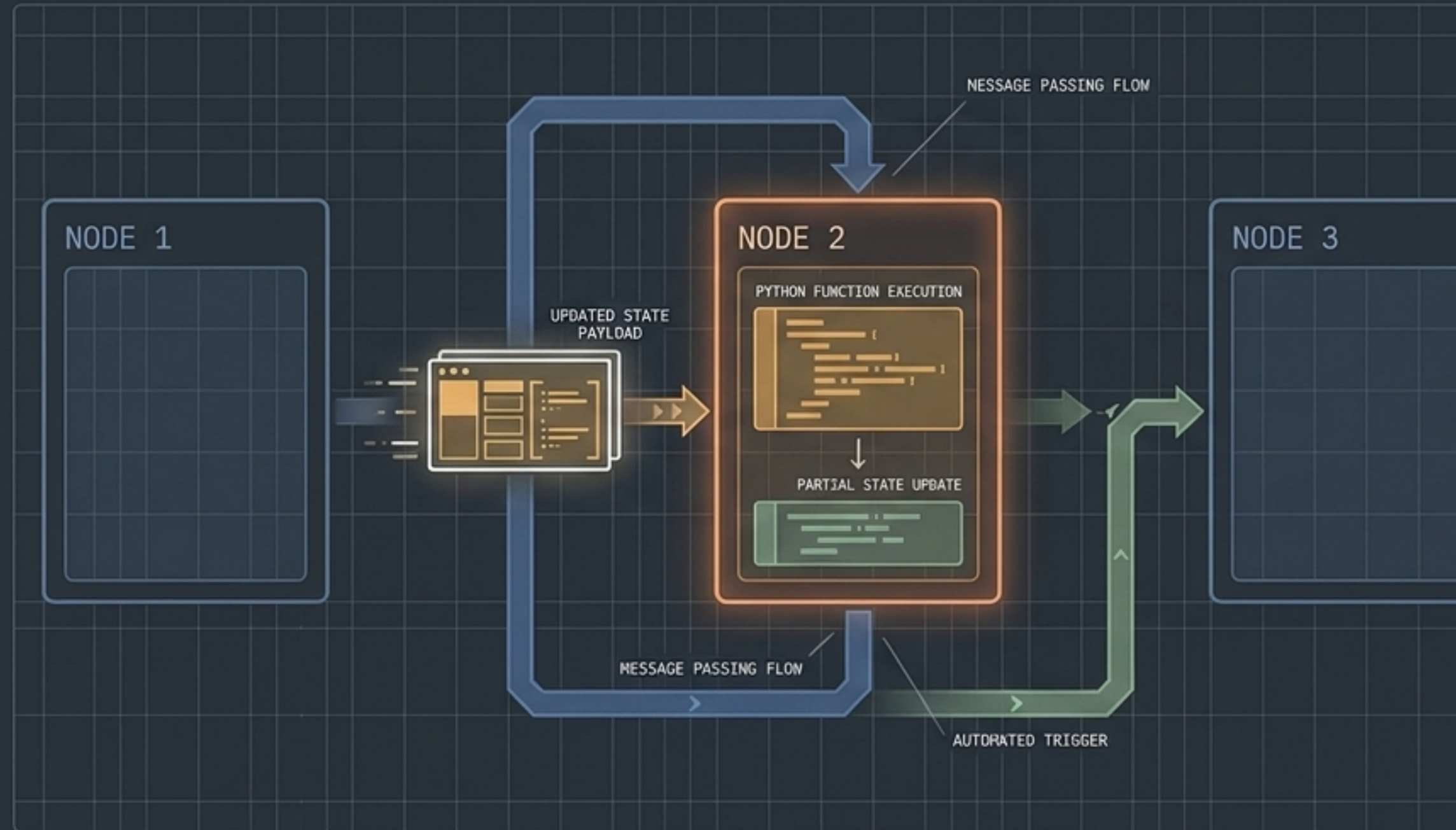


2. **Compile:** Validate structural logic. The compiler checks for fatal inconsistencies—such as **orphaned nodes** disconnected from the execution flow—ensuring the graph won't fail dynamically during runtime.



The Execution Engine: Message Passing

LangGraph does not require manually calling sequential functions. Execution is driven automatically by Message Passing.



Invoke: The initial state is passed to the starting node.

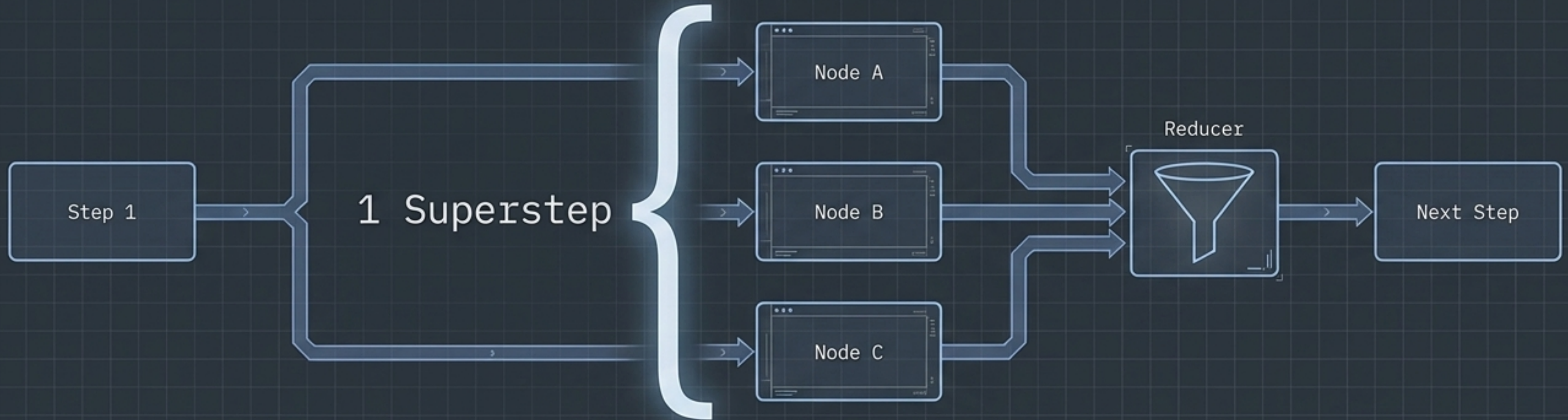
Activate: The node's Python function executes.

Update: The node makes a partial update to the state.

Pass: The updated state is passed via edges to trigger the next node downstream.

Temporal Architecture: Supersteps

Inspired by Google Pregel, LangGraph measures execution in "Supersteps" rather than simple steps.



When message passing triggers a parallel topology, multiple nodes activate simultaneously. Because they process concurrently in the same computational round, they constitute a single Superstep. The graph halts only when there are zero active nodes and zero messages passing.

The Architecture of Agency

The complete mental model for production AI. A Workflow mapped into a Graph, storing memory in a Shared State, governed by Reducers, and executed autonomously across concurrent Supersteps.

